

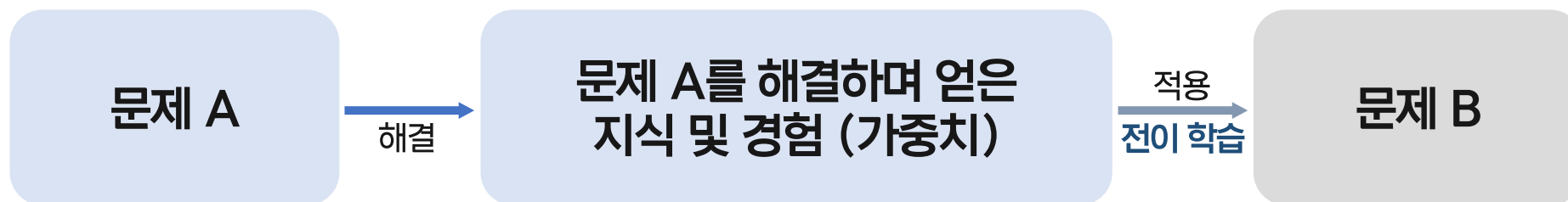
합성곱 신경망

5.3 전이학습

5.4 설명 가능한 CNN

5.5 그래프 합성곱 네트워크(GCN)

- ☑ **전이학습(Transfer Learning)**: ImageNet과 같이 매우 큰 데이터셋으로 사전훈련된 모델의 가중치를 가져와 해결하려는 과제 및 도메인에 맞게 보정하여 사용하는 방식



전이학습(Transfer Learning)

특성 추출(Feature Extractor)

사전 훈련된 모델을 가져온 후 마지막 완전연결층 부분을 새로 만들어 해당 층(분류기)만 훈련하는 방식

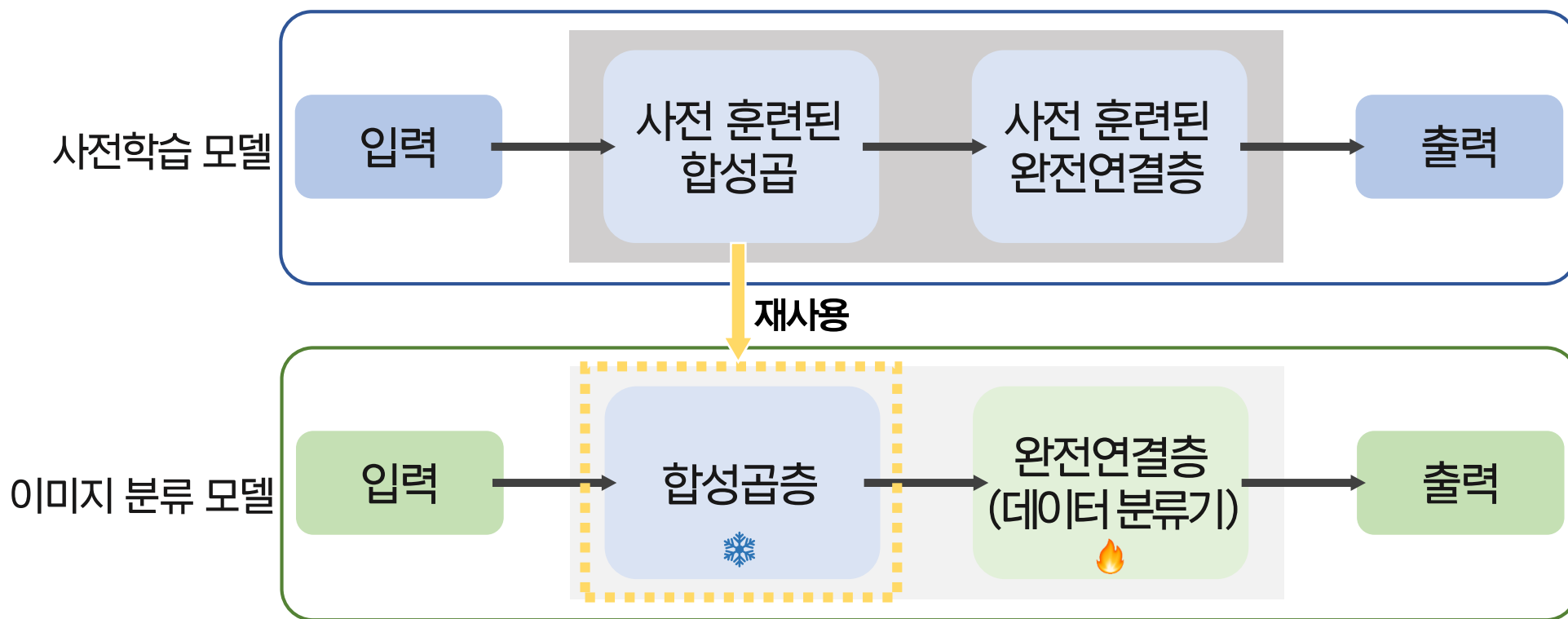
미세 조정(Fine Tuning)

사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련하는 방식
(목적에 맞게 학습된 가중치의 전체 or 일부를 재학습)

- ☑ **특성 추출(Feature Extractor):** 사전 훈련된 모델을 가져온 후 마지막 완전연결층 부분을 새로 만들어 해당 층(분류기)만 훈련하는 방식

특성 추출은 이미지 분류를 위해 합성곱층과 완전연결층(데이터 분류기)로 구성됨

- ① 합성곱층: 합성곱층과 풀링층으로 구성 → 사전훈련 네트워크의 가중치로 고정(Freeze ❄️)
- ② 완전연결층(데이터 분류기): 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류 → 보유한 데이터로 학습(🔥)

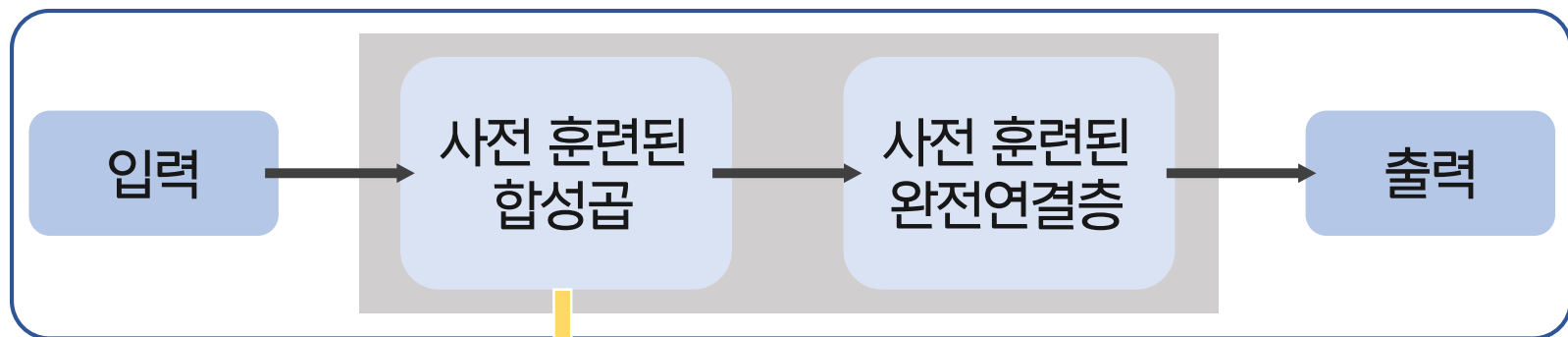


✓ **미세조정(Fine Tuning):** 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련하는 방식

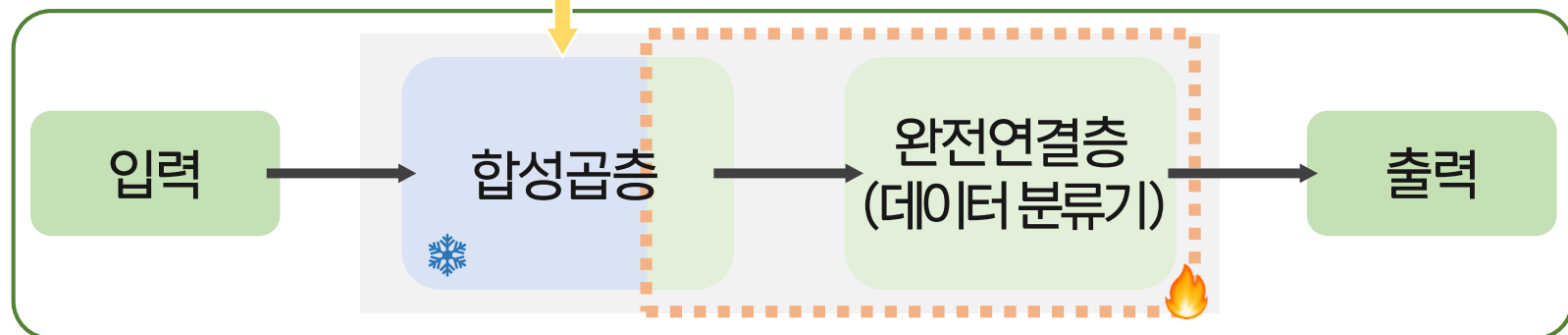
미세조정은 훈련 데이터셋의 크기와 사전훈련 모델과의 유사성에 따라 전략을 세워야 함

- ① 데이터셋 크기↑ 사전 훈련 모델과의 유사성↓
- ② 데이터셋 크기↑ 사전 훈련 모델과의 유사성↑
- ③ 데이터셋 크기↓ 사전 훈련 모델과의 유사성↓
- ④ 데이터셋 크기↓ 사전 훈련 모델과의 유사성↑

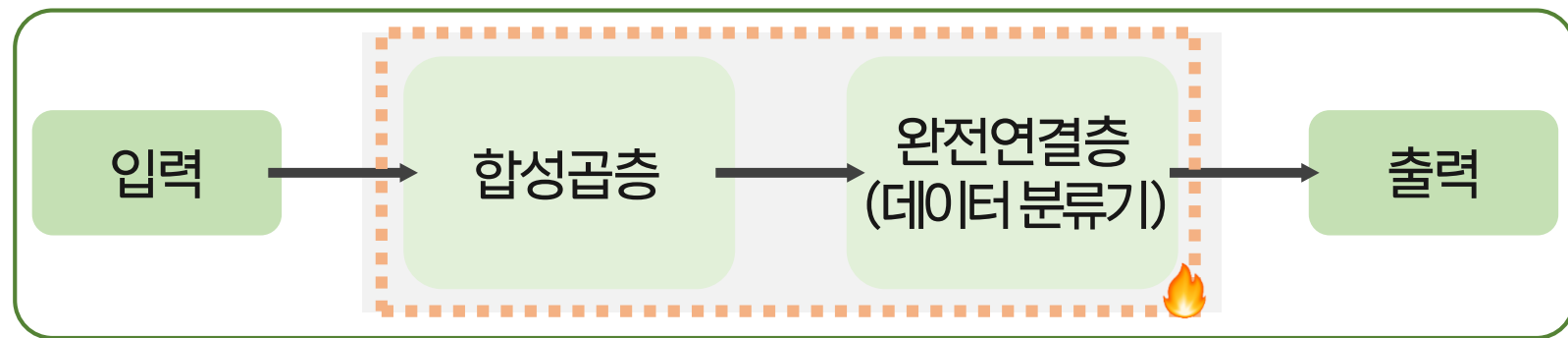
② or ③ or ④



재사용



①



☑ 전이학습(Transfer Learning) 실습 - 실험환경 구축 및 데이터 준비

```
import os
import time
import copy
import glob
import cv2
import shutil

import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim

import matplotlib.pyplot as plt
```

#컴퓨터비전 용도의 패키지 (데이터셋, 전처리, 모델 등을 제공)
#데이터 전처리 용도의 패키지
#다양한 컴퓨터비전 과제에 적용 가능한 네트워크(사전훈련모델 포함) 제공

```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload() # 데이터 불러오기: chap05/data/catndog.zip 파일 선택
```

파일 선택 catanddog.zip

catanddog.zip(application/x-zip-compressed) - 15512291 bytes, last modified: 2026. 5. 12. - 100% done
Saving catanddog.zip to catanddog.zip

```
!unzip catanddog.zip -d catanddog/ #catanddog 폴더 만들어 압축 풀기
```

```
Archive: catanddog.zip
  creating: catanddog/test/
  creating: catanddog/test/Cat/
  inflating: catanddog/test/Cat/8100.jpg
```

☑ 전이학습(Transfer Learning) 실습 - 데이터 전처리

```
data_path = 'catanddog/train/'
transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
    ])
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

385

본 실습에서의 데이터 전처리 구성

- ① Resize: 이미지의 크기 조정
- ② RandomResizedCrop: 이미지를 랜덤 크기 및 비율로 자름

```
def __init__(
    self,
    size,
    scale=(0.08, 1.0),
    ratio=(3.0 / 4.0, 4.0 / 3.0),
    interpolation=InterpolationMode.BILINEAR,
    antialias: Optional[bool] = True,
```



- ③ RandomHorizontalFlip: 이미지를 좌우 반전(50%확률)
- ④ ToTensor: Tensor로 변환

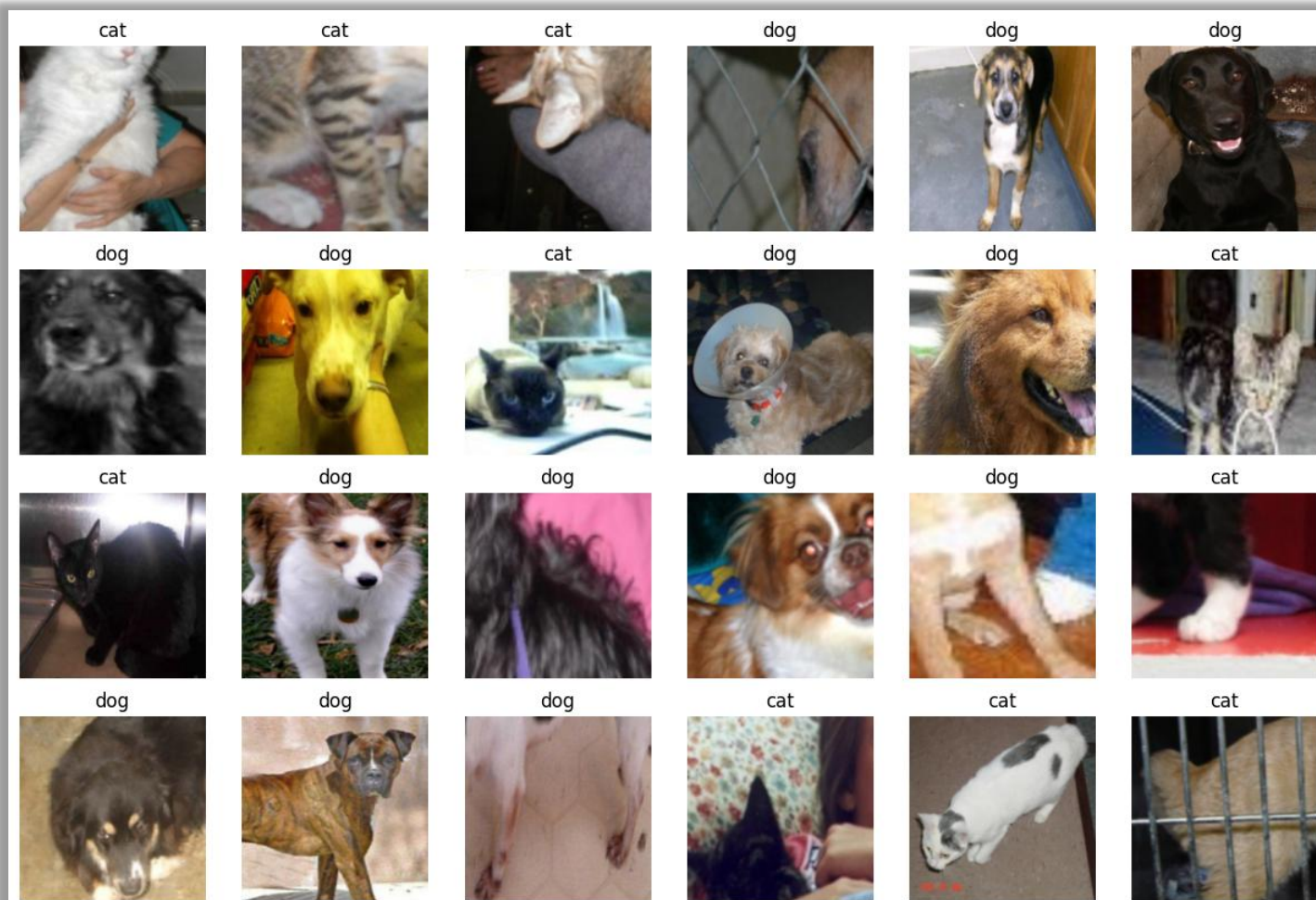
☑ 전이학습(Transfer Learning) 실습 - 데이터 확인

```
import numpy as np
samples, labels = next(iter(train_loader))
print(samples.shape)
classes = {0:'cat', 1:'dog'} #고양이/개 클래스 구성
fig = plt.figure(figsize=(15,20))
for i in range(24):
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) #클래스 정보를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432:
self.check_worker_number_rationality()
torch.Size([32, 3, 224, 224])
```

Matplotlib에서는 [height, width, channel]의 형태를 요구함

np.transpose를 통해 [3, 224, 224]→[224, 224, 3]으로 변경



☑ 전이학습(Transfer Learning) 실습 - 모델 준비

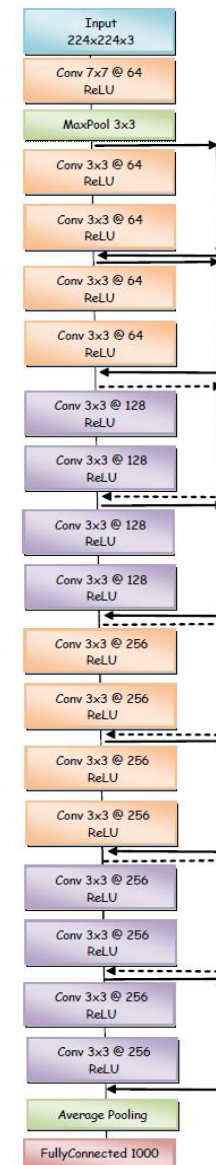
```
resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
alexnet = models.alexnet(weights=models.AlexNet_Weights.DEFAULT)
squeezenet = models.squeezenet1_0(weights=models.SqueezeNet1_0_Weights.DEFAULT)
vgg16 = models.vgg16(weights=models.VGG16_Weights.DEFAULT)
densenet = models.densenet161(weights=models.DenseNet161_Weights.DEFAULT)
inception = models.inception_v3(weights=models.Inception_V3_Weights.DEFAULT)
googlenet = models.googlenet(weights=models.GoogLeNet_Weights.DEFAULT)
shufflenet = models.shufflenet_v2_x1_0(weights=models.ShuffleNet_V2_X1_0_Weights.DEFAULT)
mobilenet_v2 = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
mobilenet_v3_large = models.mobilenet_v3_large(weights=models.MobileNet_V3_Large_Weights.DEFAULT)
mobilenet_v3_small = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.DEFAULT)
resnext50_32x4d = models.resnext50_32x4d(weights=models.ResNeXt50_32X4D_Weights.DEFAULT)
wide_resnet50_2 = models.wide_resnet50_2(weights=models.Wide_ResNet50_2_Weights.DEFAULT)
mnasnet = models.mnasnet1_0(weights=models.MNASNet1_0_Weights.DEFAULT)
```

사전 학습된 ResNet-18 모델로 실습 진행

```
(avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
(fc): Linear(in_features=512, out_features=1000, bias=True)
```

```
def set_parameter_requires_grad(model, feature_extracting=True):
    # feature extraction 방식이면 기존 사전학습 파라미터를 고정
    if feature_extracting:
        # 모델 안의 모든 파라미터를 하나씩 가져옴
        for param in model.parameters():
            # gradient 계산을 막아서 학습 중 업데이트되지 않게 함
            param.requires_grad = False
```

ResNet-18



✓ 전이학습(Transfer Learning) 실습 - 학습/검증 함수

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=30, is_train=True, save_dir='catanddog/checkpoints'):
    since = time.time()
    acc_history = []
    loss_history = []
    best_acc = 0.0

    os.makedirs(save_dir, exist_ok=True)

    for epoch in range(num_epochs): # 전체 데이터셋을 num_epochs만큼 반복 학습
        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # dataloader에서 mini-batch 단위로 이미지와 라벨을 가져옴
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad() # 이전 batch에서 계산된 gradient를 초기화
            outputs = model(inputs)
            loss = criterion(outputs, labels) # 예측값과 실제 label을 비교하여 loss 계산
            _, preds = torch.max(outputs, 1) # outputs에서 가장 큰 값을 가진 클래스 index를 예측 label로 선택
            loss.backward() # Backward propagation loss를 기준으로 각 가중치 gradient 계산
            optimizer.step() # Optimizer가 gradient를 이용해 모델 가중치 업데이트

            running_loss += loss.item() * inputs.size(0) # 현재 batch의 loss를 전체 loss에 누적
            running_corrects += torch.sum(preds == labels.data) # 현재 batch에서 맞게 예측한 개수 누적

        epoch_loss = running_loss / len(dataloaders.dataset) # 한 epoch의 평균 loss 계산
        epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 한 epoch의 accuracy 계산

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        torch.save(model.state_dict(), os.path.join(save_dir, '{0:0=2d}.pth'.format(epoch))) # 현재 epoch의 모델 가중치 저장
        print()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))
    return acc_history, loss_history
```

```
def eval_model(model, dataloaders, device, load_dir='catanddog/'):
    since = time.time()
    acc_history = []
    best_acc = 0.0
    saved_models = glob.glob(os.path.join(load_dir, '*.pth')) # load_dir 폴더 안에 있는 .pth 모델 파일들을 모두 찾음
    saved_models.sort()
    print('saved_model', saved_models)

    for model_path in saved_models: # 저장된 모델 파일을 하나씩 불러와 평가
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path)) # train_model에서 저장한 state_dict를 다시 로드하는 과정
        model.eval() # 모델을 평가 모드로 전환(Dropout과 같은 레이어가 평가용으로 동)
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # dataloader에서 batch 단위로 이미지와 정답 label을 가져옴
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # 평가 단계에서는 gradient 계산이 필요 없음
                outputs = model(inputs)

            _, preds = torch.max(outputs.data, 1) # outputs에서 가장 큰 값을 가진 클래스 index를 예측값으로 선택
            preds[preds >= 0.5] = 1
            preds[preds < 0.5] = 0
            running_corrects += preds.eq(labels).int().sum() # 예측값과 실제 label이 같은 개수를 누적

        epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 전체 데이터셋 기준 accuracy 계산
        print('Acc: {:.4f}'.format(epoch_acc)) # 현재 저장 모델의 accuracy 출력

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history
```

✓ 전이학습(Transfer Learning) 실습 - 특성 추출(Feature Extractor) 실습

```
FE_resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

```
# 모델의 기존 파라미터들을 모두 고정
set_parameter_requires_grad(FE_resnet18)
```

```
for name, param in FE_resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)
```

```
model = FE_resnet18
```

```
for param in model.parameters():
    param.requires_grad = False
```

```
model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True
```

```
optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss()
print(model)
```

```
(avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
(fc): Linear(in_features=512, out_features=2, bias=True)
)
```

```
params_to_update = []
for name, param in FE_resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
        print("\t", name)

optimizer = optim.Adam(params_to_update)
```

```
fc.weight
fc.bias
```

☑ 전이학습(Transfer Learning) 실습 - 특성 추출(Feature Extractor) 실습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
FE_train_acc_hist, FE_train_loss_hist = train_model(FE_resnet18, train_loader, criterion, optimizer, device, save_dir='catanddog/FE')
```

Epoch 23/29

Loss: 0.1387 Acc: 0.9610

Epoch 24/29

Loss: 0.2246 Acc: 0.9169

Epoch 25/29

Loss: 0.1921 Acc: 0.9195

Epoch 26/29

Loss: 0.1775 Acc: 0.9299

Epoch 27/29

Loss: 0.1618 Acc: 0.9299

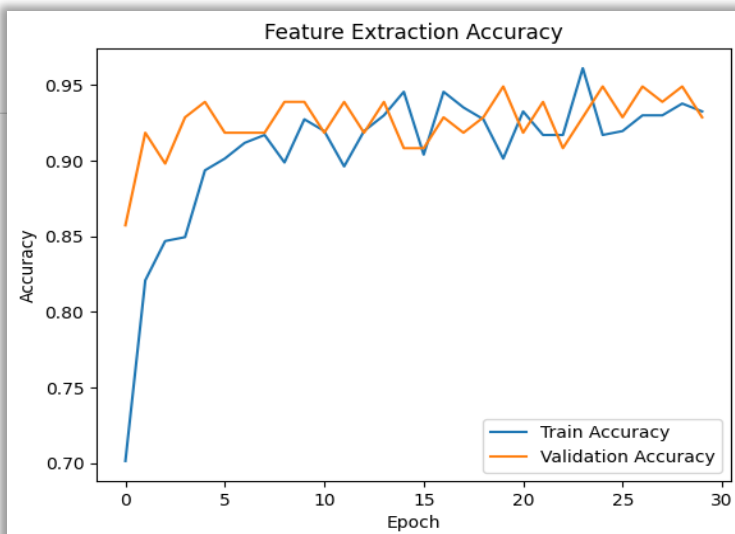
Epoch 28/29

Loss: 0.1577 Acc: 0.9377

Epoch 29/29

Loss: 0.1715 Acc: 0.9325

Training complete in 1m 25s
Best Acc: 0.961039



```
FE_val_acc_hist = eval_model(FE_resnet18, test_loader, device, load_dir='catanddog/FE')
```

Loading model catanddog/FE/23.pth
Acc: 0.9286

Loading model catanddog/FE/24.pth
Acc: 0.9490

Loading model catanddog/FE/25.pth
Acc: 0.9286

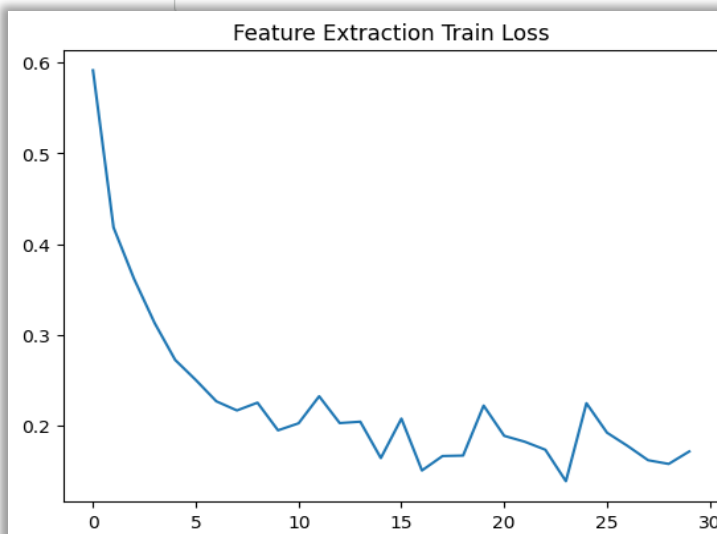
Loading model catanddog/FE/26.pth
Acc: 0.9490

Loading model catanddog/FE/27.pth
Acc: 0.9388

Loading model catanddog/FE/28.pth
Acc: 0.9490

Loading model catanddog/FE/29.pth
Acc: 0.9286

Validation complete in 0m 19s
Best Acc: 0.948980



✓ 전이학습(Transfer Learning) 실습 - 특성 추출(Feature Extractor) 실습

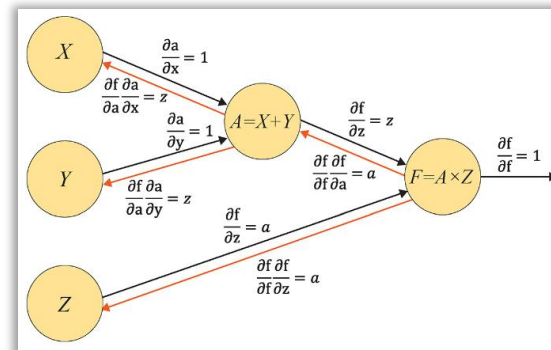
```
def im_convert(tensor): # PyTorch Tensor 이미지를 imshow가 볼 수 있는 형태로 바꾸는 함수
    image = tensor.clone().detach().cpu().numpy()
    image = image.transpose(1, 2, 0)
    image = image.clip(0, 1)
    return image
```

```
classes = {0:'cat', 1:'dog'} # 클래스 번호를 실제 클래스 이름으로 매핑
```

```
dataiter=iter(test_loader) # test_loader를 iterator로 변환
images, labels = next(dataiter) # test_loader에서 첫 번째 batch를 꺼냄
images = images.to(device)
labels = labels.to(device)
model=FE_resnet18 # 평가에 사용할 모델 지정
output=model(images) # 이미지 batch를 모델에 입력하여 예측 결과 계산
_,preds=torch.max(output,1) # output에서 가장 큰 점수를 가진 클래스 index를 예측값으로 선택 ( _:최댓값, preds:예측 클래스)
```

```
fig=plt.figure(figsize=(25,4))
for idx in np.arange(20): # batch 안의 이미지 중 앞에서부터 20장을 시각화
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])
    plt.imshow(im_convert(images[idx]))
    a.set_title(classes[labels[i].item()])
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()])),color=("green" if preds[idx]==labels[idx] else "red"))
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

구분	메모리	계산 그래프 상주 여부	의미
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주	원본 tensor와 같은 값을 가진 새 tensor를 만들지만, gradient 계산 흐름은 유지됨
tensor.detach()	공유해서 사용	계산 그래프에서 상주하지 않음	원본 tensor와 같은 메모리를 공유하지만, gradient 계산에서는 분리됨
tensor.clone().detach()	새롭게 할당	계산 그래프에서 상주하지 않음	값을 복사해서 새 tensor를 만들고, gradient 계산에서도 완전히 분리함



✓ 전이학습(Transfer Learning) 실습 - 미세 조정(Fine Tuning) 실습

```
FT_resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

```
# 모델의 기존 파라미터들을 고정 해제
set_parameter_requires_grad(FT_resnet18, feature_extracting=False)
```

```
model = FT_resnet18
```

```
for param in model.parameters():
    param.requires_grad = True
```

```
model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True
```

```
optimizer = torch.optim.Adam(model.parameters())
cost = torch.nn.CrossEntropyLoss()
print(model)
```

```
params_to_update = []
for name,param in FT_resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
        print("\t",name)

optimizer = optim.Adam(params_to_update)
```

```
conv1.weight
bn1.weight
bn1.bias
layer1.0.conv1.weight
layer1.0.bn1.weight
layer1.0.bn1.bias
layer1.0.conv2.weight
layer1.0.bn2.weight
layer1.0.bn2.bias
layer1.1.conv1.weight
layer1.1.bn1.weight
layer1.1.bn1.bias
layer1.1.conv2.weight
layer1.1.bn2.weight
layer1.1.bn2.bias
layer2.0.conv1.weight
layer2.0.bn1.weight
layer2.0.bn1.bias
layer2.0.conv2.weight
layer2.0.bn2.weight
layer2.0.bn2.bias
layer2.0.downsample.0.weight
layer2.0.downsample.1.weight
layer2.0.downsample.1.bias
layer2.1.conv1.weight
layer2.1.bn1.weight
layer2.1.bn1.bias
layer2.1.conv2.weight
layer2.1.bn2.weight
layer2.1.bn2.bias
layer3.0.conv1.weight
layer3.0.bn1.weight
layer3.0.bn1.bias
layer3.0.conv2.weight
layer3.0.bn2.weight
layer3.0.bn2.bias
layer3.0.downsample.0.weight
layer3.0.downsample.1.weight
layer3.0.downsample.1.bias
layer3.1.conv1.weight
layer3.1.bn1.weight
layer3.1.bn1.bias
layer3.1.conv2.weight
layer3.1.bn2.weight
layer3.1.bn2.bias
layer4.0.conv1.weight
layer4.0.bn1.weight
layer4.0.bn1.bias
layer4.0.conv2.weight
layer4.0.bn2.weight
layer4.0.bn2.bias
layer4.0.downsample.0.weight
layer4.0.downsample.1.weight
layer4.0.downsample.1.bias
layer4.1.conv1.weight
layer4.1.bn1.weight
layer4.1.bn1.bias
layer4.1.conv2.weight
layer4.1.bn2.weight
layer4.1.bn2.bias
fc.weight
fc.bias
```

☑ 전이학습(Transfer Learning) 실습 - 미세 조정(Fine Tuning) 실습

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
FT_train_acc_hist, FT_train_loss_hist = train_model(FT_resnet18, train_loader, criterion, optimizer, device, save_dir='catanddog/FT')
```

```
Epoch 22/29
-----
Loss: 0.3651 Acc: 0.8390

Epoch 23/29
-----
Loss: 0.2699 Acc: 0.8753

Epoch 24/29
-----
Loss: 0.3102 Acc: 0.8597

Epoch 25/29
-----
Loss: 0.3538 Acc: 0.8597

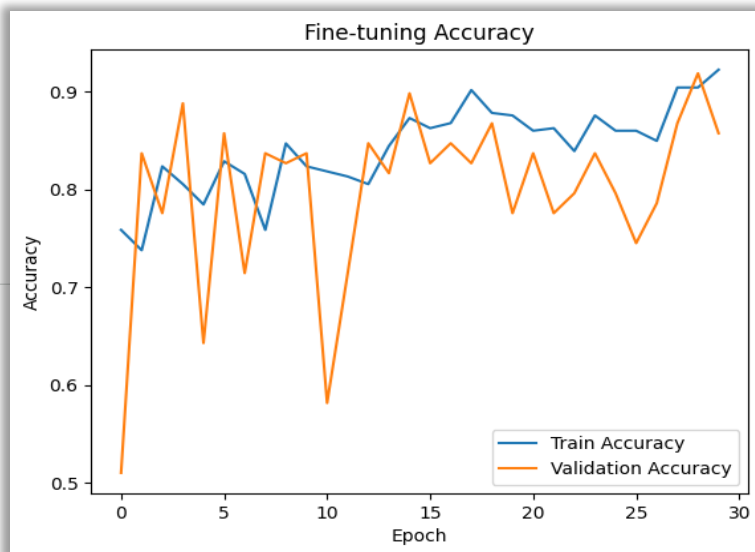
Epoch 26/29
-----
Loss: 0.3204 Acc: 0.8494

Epoch 27/29
-----
Loss: 0.2677 Acc: 0.9039

Epoch 28/29
-----
Loss: 0.2226 Acc: 0.9039

Epoch 29/29
-----
Loss: 0.2067 Acc: 0.9221

Training complete in 1m 43s
Best Acc: 0.922078
```



```
FT_val_acc_hist = eval_model(FT_resnet18, test_loader, device, load_dir='catanddog/FT')
```

```
Loading model catanddog/FT/24.pth
Acc: 0.7959

Loading model catanddog/FT/25.pth
Acc: 0.7449

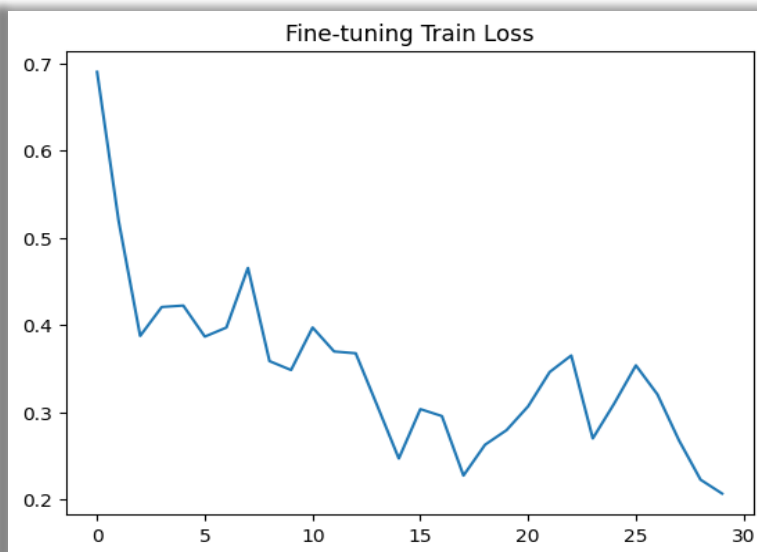
Loading model catanddog/FT/26.pth
Acc: 0.7857

Loading model catanddog/FT/27.pth
Acc: 0.8673

Loading model catanddog/FT/28.pth
Acc: 0.9184

Loading model catanddog/FT/29.pth
Acc: 0.8571

Validation complete in 0m 19s
Best Acc: 0.918367
```

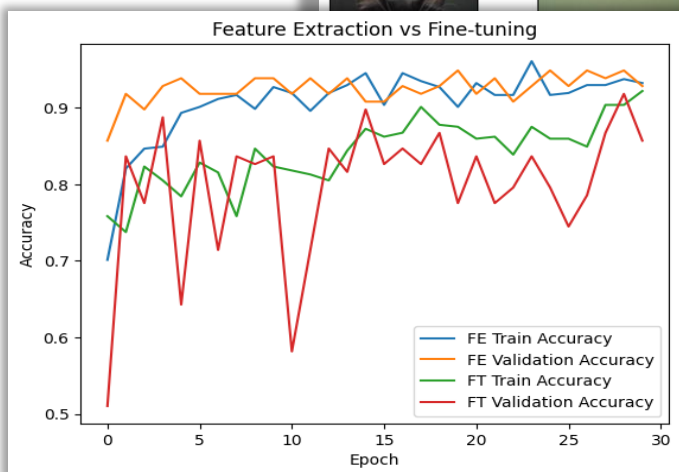


☑ 전이학습(Transfer Learning) 실습 - 특성 추출(Feature Extractor) vs 미세 조정(Fine Tuning)

특성 추출
(Feature Extractor)



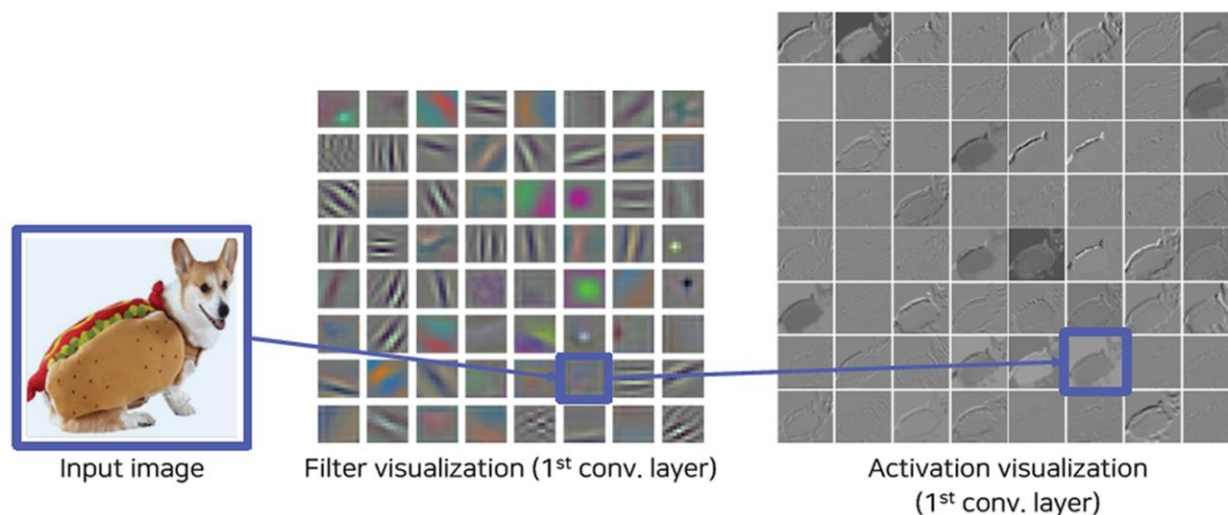
미세 조정
(Fine Tuning)



- ☑ **설명 가능한 CNN(explainable CNN):** 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 처리하는 기술
 CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 특성이 어떻게 추출되고 학습됐는지
 시각적으로 설명함으로써 결과에 대한 신뢰성을 얻을 수 있음



▶ **필터에 대한 시각화와 특성 맵에 대한 시각화를 통해 설명 가능**



✓ 설명 가능한 CNN(explainable CNN) 실습 - XAI 모델 준비

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(128, 128, kernel_size=10, padding=1, bias=False),
nn.BatchNorm2d(128),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(256),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.Dropout(0.4),
nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
nn.BatchNorm2d(512),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=2, stride=2),
```

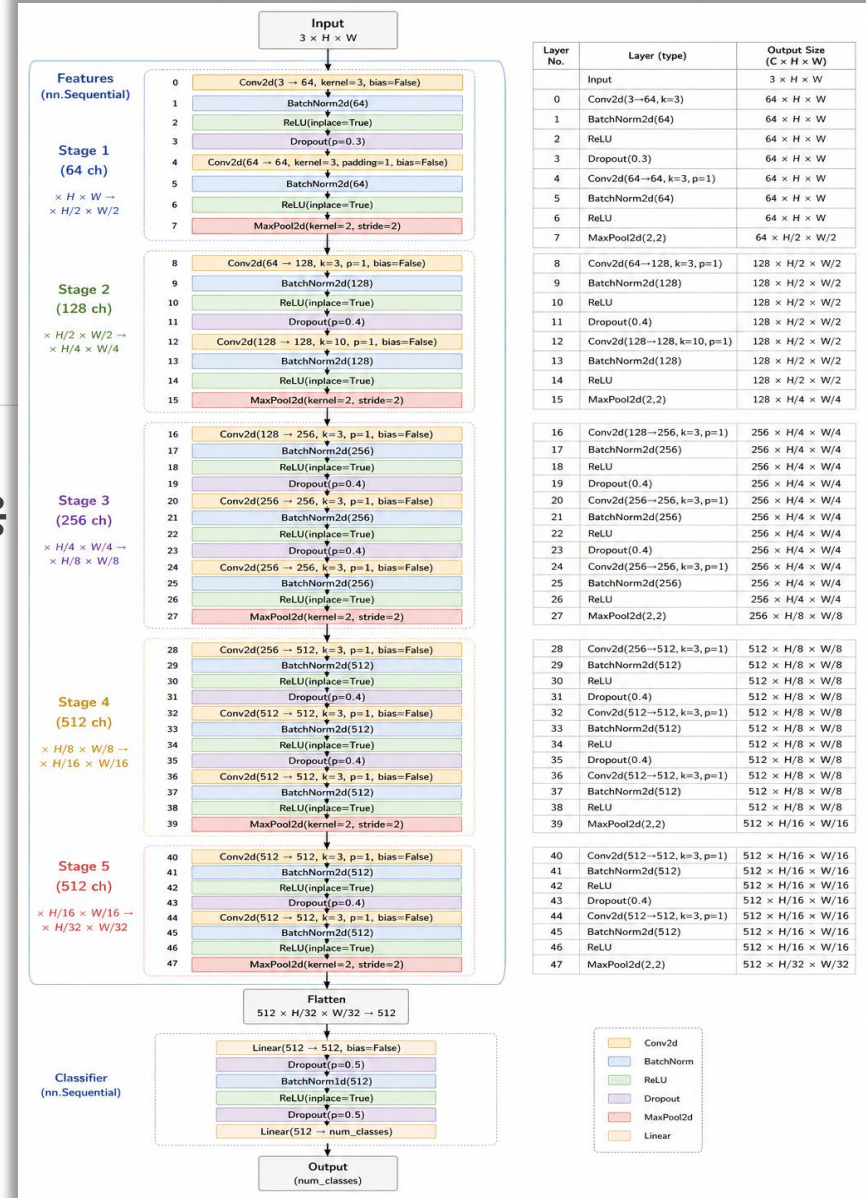
```
self.classifier = nn.Sequential(
    nn.Linear(512, 512, bias=False),
    nn.Dropout(0.5),
    nn.BatchNorm1d(512),
    nn.ReLU(inplace=True),
    nn.Dropout(0.5),
    nn.Linear(512, num_classes)
)
```

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x, dim=1)
```

기울기 소멸 문제에 취약한 소프트맥스의 단점을 보완하기 위해 로그 소프트맥스 적용

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

$$\text{logsoftmax}(x)_i = x_i - \log\left(\sum_j e^{x_j}\right)$$



✓ 설명 가능한 CNN(explainable CNN) 실습 - 필터/특성 맵 추출 함수 준비

```
model=XAI()
model.cpu() #모델에 입력되는 이미지를 넘파이로 받아오는 부분때문에 CPU를 사용하도록 지정
model.eval()
```

```
XAI(
    (features): Sequential(
      (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): Dropout(p=0.3, inplace=False)
      (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (6): ReLU(inplace=True)
      (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (10): ReLU(inplace=True)
      (11): Dropout(p=0.4, inplace=False)
      (12): Conv2d(128, 128, kernel_size=(10, 10), stride=(1, 1), padding=(1, 1), bias=False)
      (13): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (14): ReLU(inplace=True)
      (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (16): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (18): ReLU(inplace=True)
      (19): Dropout(p=0.4, inplace=False)
      (20): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (22): ReLU(inplace=True)
      (23): Dropout(p=0.4, inplace=False)
      (24): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (25): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (26): ReLU(inplace=True)
      (27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (28): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (29): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (30): ReLU(inplace=True)
      (31): Dropout(p=0.4, inplace=False)
      (32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (33): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (34): ReLU(inplace=True)
      (35): Dropout(p=0.4, inplace=False)
      (36): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (38): ReLU(inplace=True)
      (39): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
      (40): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (41): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (42): ReLU(inplace=True)
      (43): Dropout(p=0.4, inplace=False)
      (44): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (45): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (46): ReLU(inplace=True)
      (47): Dropout(p=0.4, inplace=False)
      (48): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (49): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (50): ReLU(inplace=True)
      (51): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
)
```

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num): # 모델이 forward를 수행할 때 해당 layer의 입력과 출력을 가져옴
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        output = output # output은 해당 layer의 activation 결과 (특성맵)
        #self.features = output.to(device).detach().numpy()
        self.features = output.detach().numpy()

    def remove(self): # hook을 계속 남겨두면 이후 forward 때마다 계속 실행되므로, 등록한 hook을 제거하는 함수
        self.hook.remove()
```

```
class LayerFilters:
    def __init__(self, model, layer_num):
        self.layer = model[layer_num] # model[layer_num]에 해당하는 layer를 가져옴

    # 해당 layer가 Conv2d인지 확인
    if not isinstance(self.layer, torch.nn.Conv2d):
        raise TypeError("선택한 layer는 Conv2d layer가 아닙니다.")

    self.filters = self.layer.weight.detach().cpu().numpy() # Conv2d layer의 filter weight를 가져옴
    # weight shape: [out_channels, in_channels, kernel_height, kernel_width]

    def show(self, num_filters=16): # 지정한 개수만큼 filter 시각화
        plt.figure(figsize=(12, 8))

        for idx in range(num_filters):
            ax = plt.subplot(4, 4, idx + 1)
            ax.set_xticks([])
            ax.set_yticks([])

            # idx번째 filter 선택
            filter_img = self.filters[idx]

            # 입력 채널이 3개면 RGB filter처럼 시각화 가능
            if filter_img.shape[0] == 3:
                # [C, H, W] -> [H, W, C]
                filter_img = filter_img.transpose(1, 2, 0)

            else:
                filter_img = filter_img.mean(axis=0)

            # filter 값은 음수-양수일 수 있으므로 0~1 범위로 정규화
            filter_img = (filter_img - filter_img.min()) / (filter_img.max() - filter_img.min() + 1e-8)

            if filter_img.ndim == 3: # RGB면 컬러로, 2D면 gray로 출력
                ax.imshow(filter_img)
            else:
                ax.imshow(filter_img, cmap='gray')

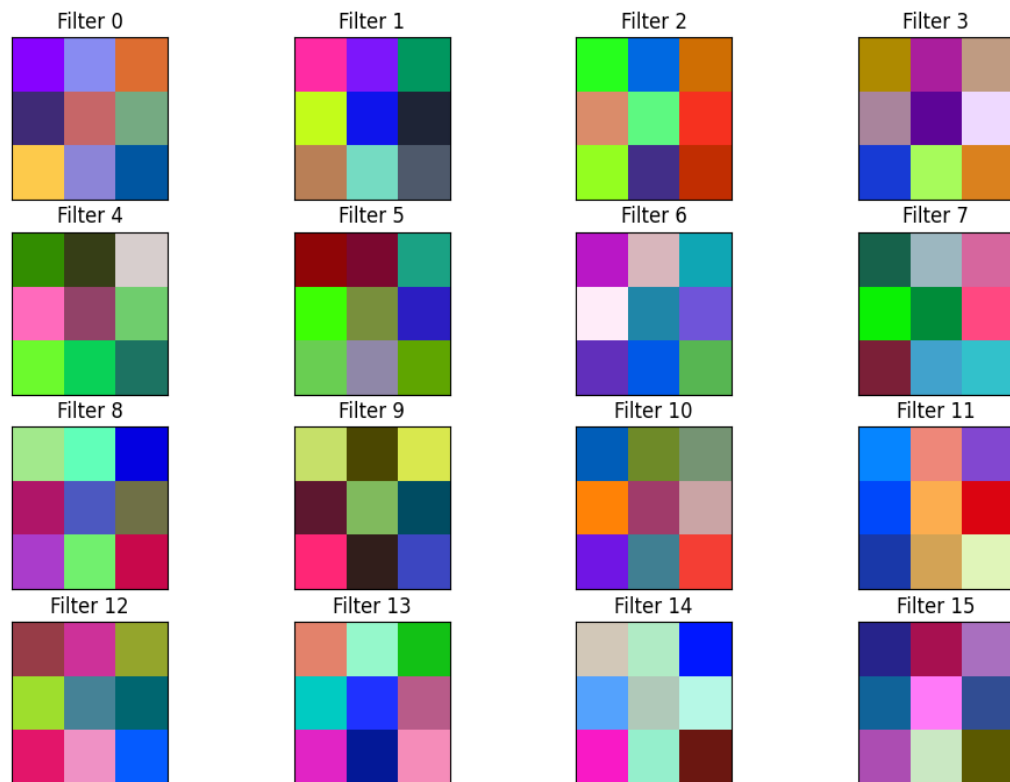
            ax.set_title(f"Filter {idx}")

        plt.suptitle("Convolution Filters")
        plt.show()
```

✓ 설명 가능한 CNN(explainable CNN) 실습 - 필터 시각화

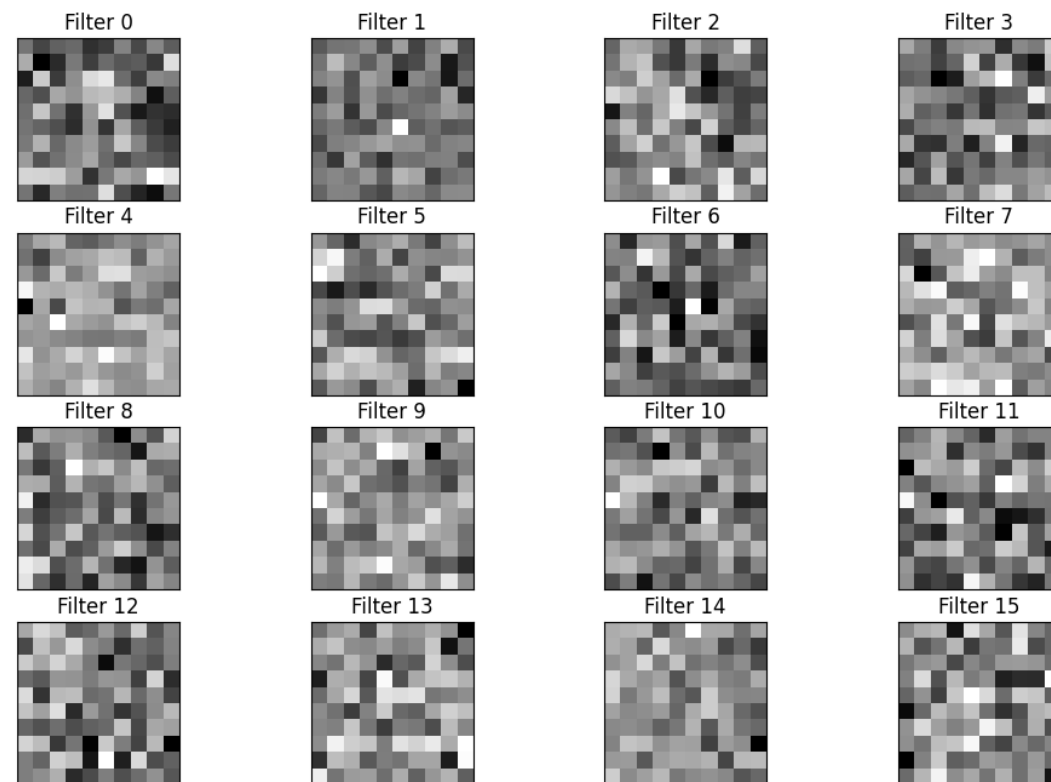
```
filter_vis = LayerFilters(model.features, 0)
filter_vis.show(num_filters=16)
```

Convolution Filters



```
filter_vis = LayerFilters(model.features, 12)
filter_vis.show(num_filters=16)
```

Convolution Filters



✓ 설명 가능한 CNN(explainable CNN) 실습 - 특성 맵 시각화

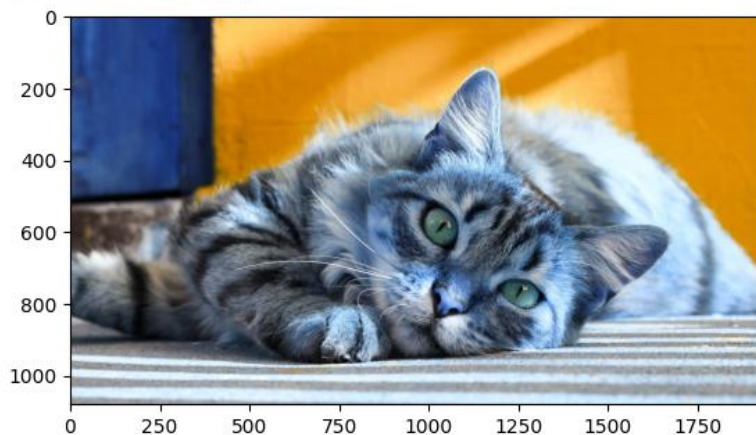
```
from google.colab import files # 데이터 불러오기
file_uploaded=files.upload() # chap05/data/cat.jpg 데이터 불러오기
```

```
img=cv2.imread("cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)
img = ToTensor()(img).unsqueeze(0)
```

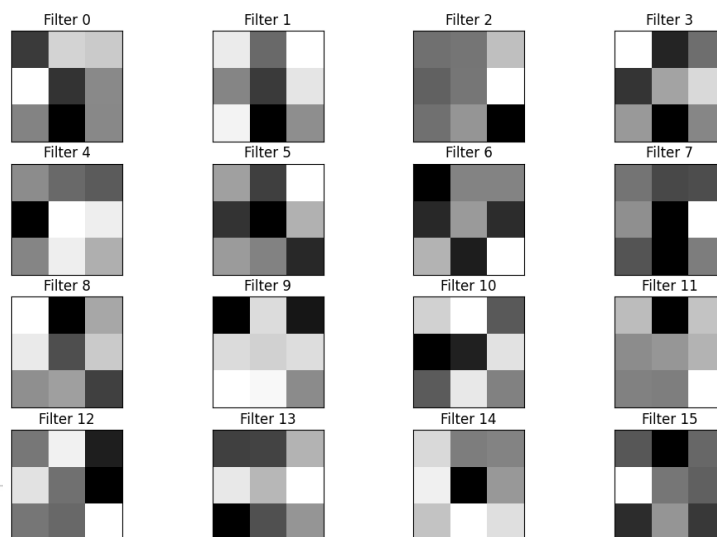
```
print(img.shape)
```

파일 선택 cat.jpg

cat.jpg(image/jpeg) - 216293 bytes, last modified: 2026. 5. 13. - 100% done
Saving cat.jpg to cat (2).jpg
torch.Size([1, 3, 100, 100])

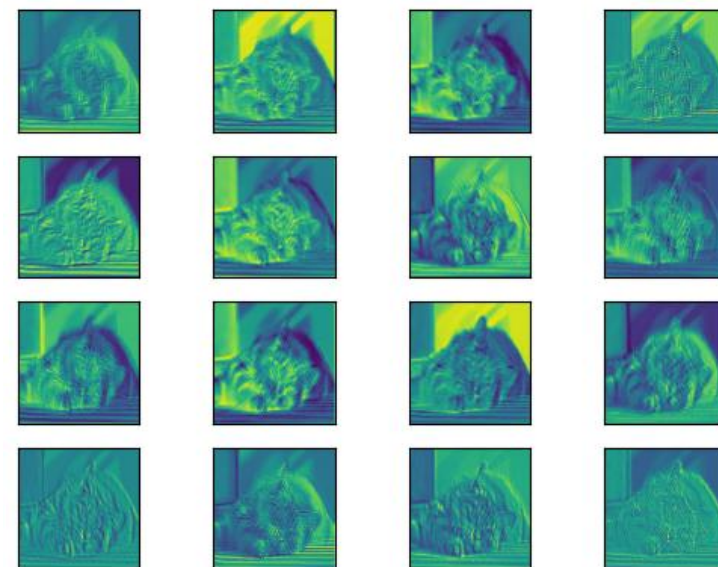


Convolution Filters



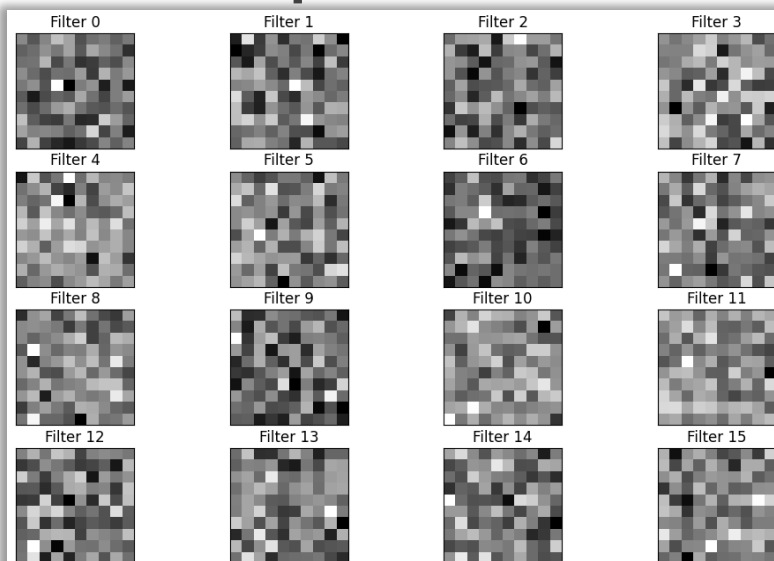
```
result = LayerActivations(model.features, 4)
model(img)
activations = result.features
```

```
fig, axes = plt.subplots(4,4)
fig = plt.figure(figsize=(12, 8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

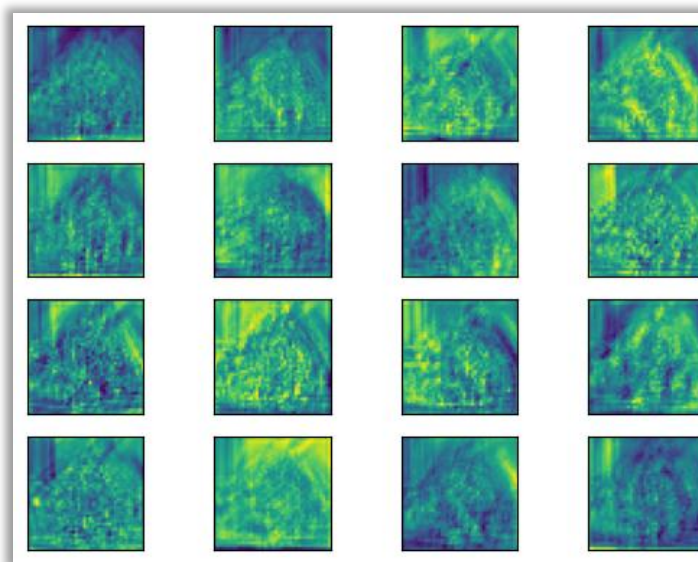
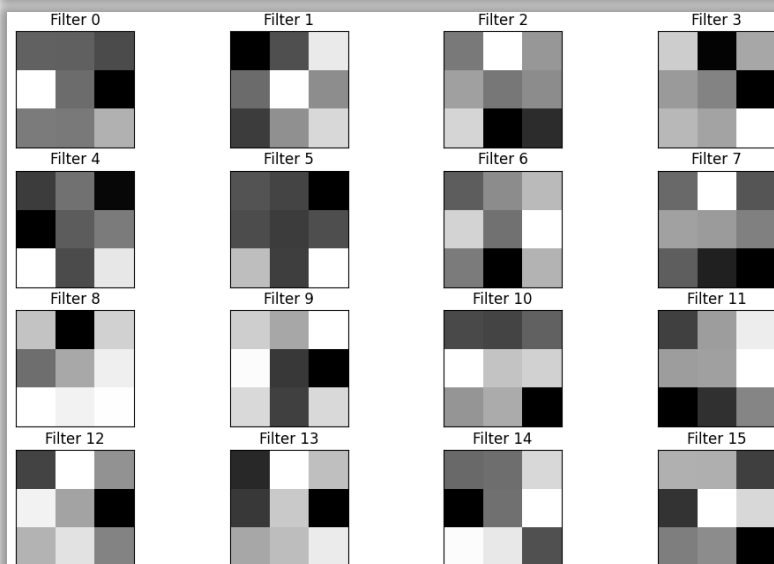


☑ 설명 가능한 CNN(explainable CNN) 실습 - 특성 맵 시각화

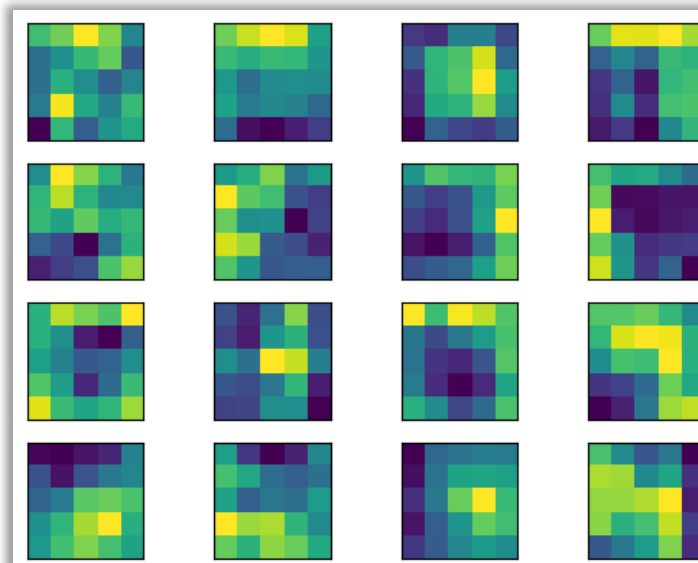
Layer 12
필터 → 특성 맵



Layer 48
필터 → 특성 맵

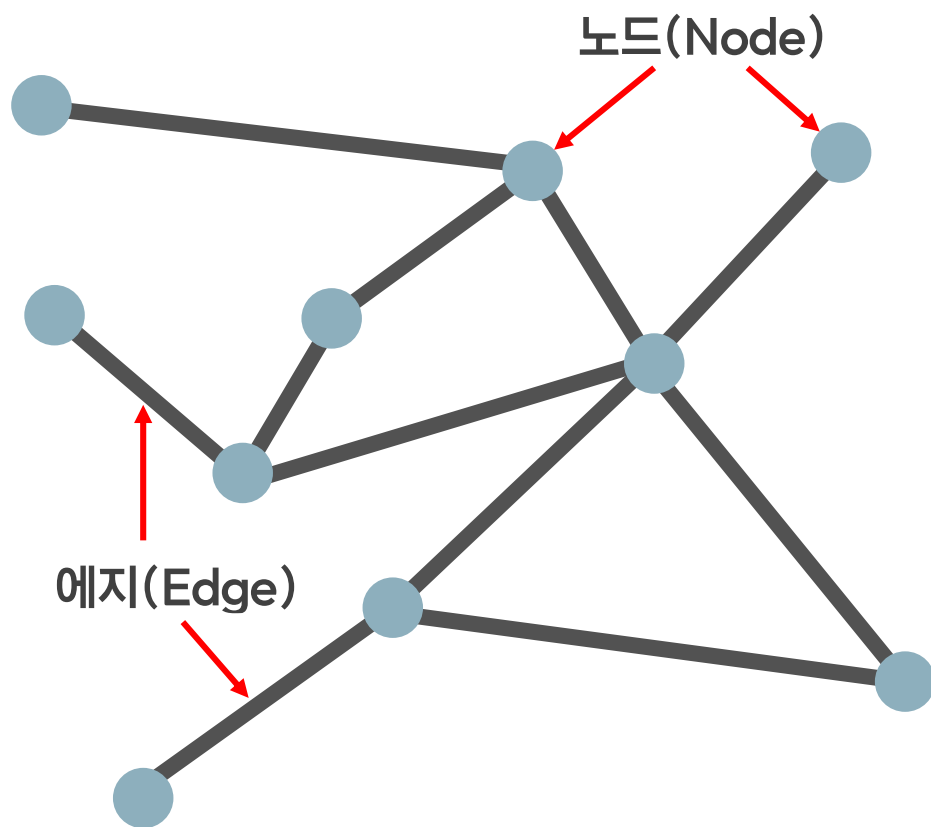


출력 층에 가까워질수록
원래 형태는 찾아볼 수 없고,
이미지 특징들만 전달됨.



특성 맵을 통해 특정 클래스
예측에 주요하게 작용한 영역을
GradCAM을 통해 설명 가능

- ☑ **그래프 합성곱 네트워크(Graph Convolution Network):** 그래프 구조 분석을 위한 합성곱 신경망
 그래프란 에지(edge)로 연결된 노드(node)의 집합(방향성 존재 가능)을 말함



그래프 데이터에 대한 표현은 두 단계로 이루어짐

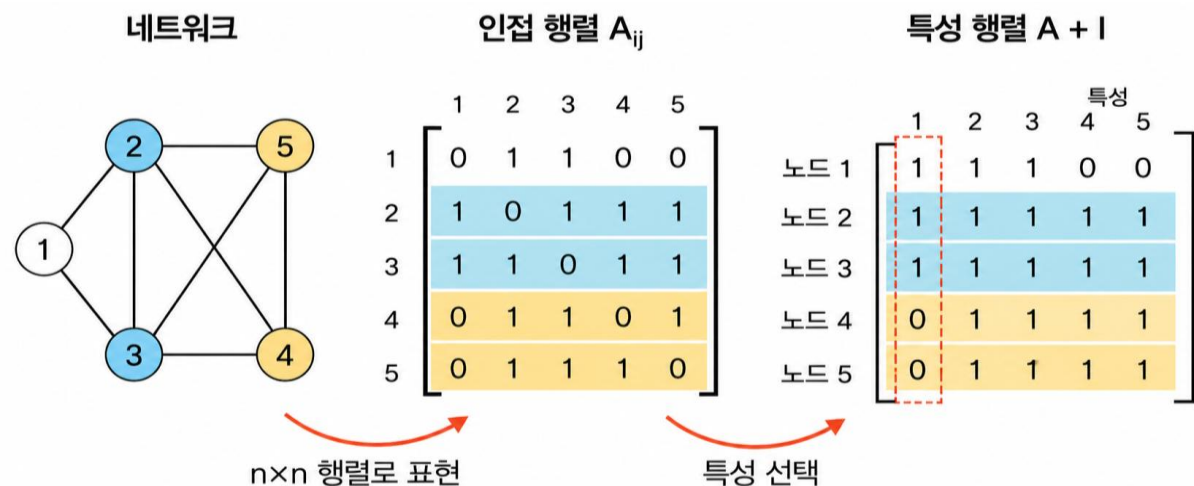
① 인접 행렬(adjacency matrix)

그래프를 $n \times n$ 행렬로 표현

② 특성 행렬(feature matrix)

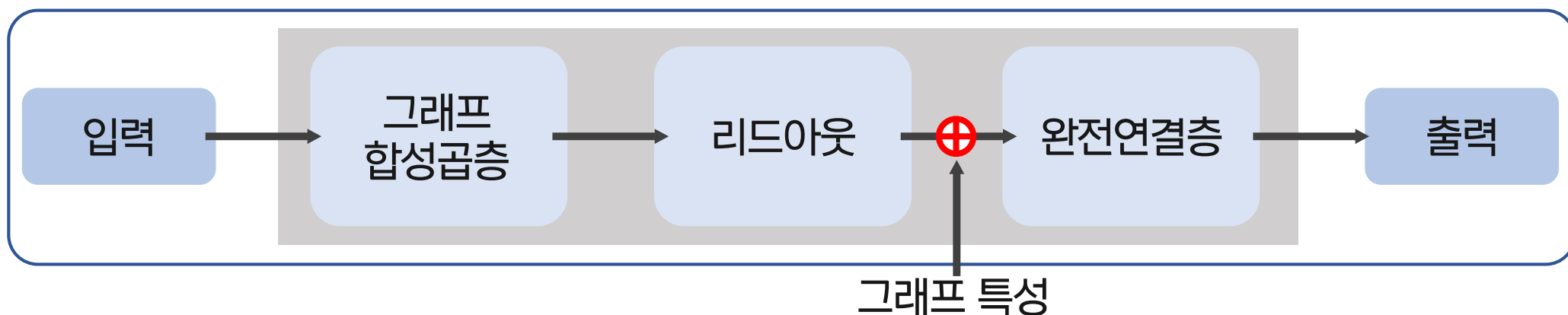
단위행렬을 적용해 각 입력데이터에서 이용할 특성 선택

각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미



☑ 그래프 합성곱 네트워크(Graph Convolution Network): 그래프 구조 분석을 위한 합성곱 신경망

그래프 합성곱 신경망은 다음과 같은 구조를 가짐



이때, 리드아웃은 특성 행렬을 하나의 벡터로 변환하는 함수

☑ 그래프 합성곱 네트워크(Graph Convolution Network) 실습: 데이터 준비 및 전처리

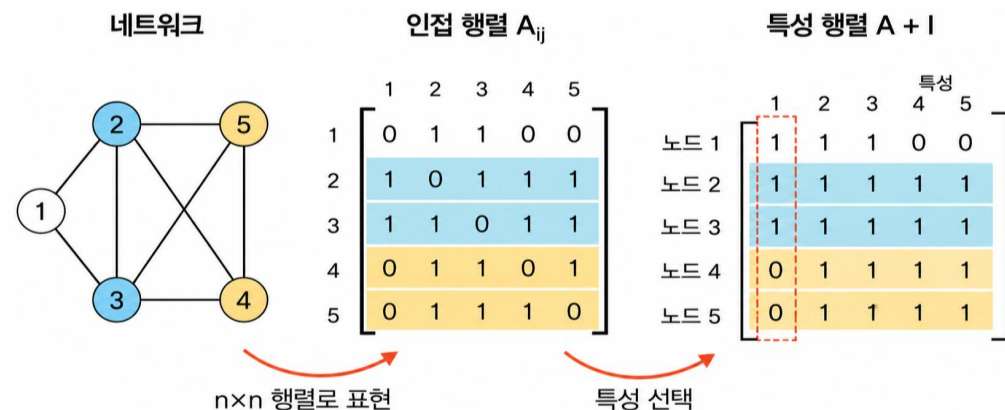
```
# 노드 수
num_nodes = 5

# 인접행렬 A
# A[i, j] = 1이면 i번 노드와 j번 노드가 연결되어 있다는 의미
A = torch.tensor([
    [0, 1, 1, 0, 0],
    [1, 0, 1, 1, 1],
    [1, 1, 0, 1, 1],
    [0, 1, 1, 0, 1],
    [0, 1, 1, 1, 0]
], dtype=torch.float32)

# 노드 특성 행렬 X
# 각 노드가 가진 feature
# shape: [노드 수, 노드 feature 차원]
X = torch.tensor([
    [1.0, 0.0, 0.5],
    [0.8, 0.2, 0.4],
    [0.9, 0.1, 0.6],
    [0.1, 0.9, 0.3],
    [0.2, 0.8, 0.2]
], dtype=torch.float32)

# 그래프 전체 특성
# shape: [그래프 feature 차원]
graph_feature = torch.tensor([0.7, 0.3], dtype=torch.float32)

# 그래프 label
# graph classification 예제이므로 그래프 전체에 label 하나만 있음
label = torch.tensor([0], dtype=torch.long)
```



```
# 자기 자신과의 연결을 추가하기 위한 단위행렬 I
I = torch.eye(num_nodes)

# A + I
# GCN에서는 이웃 노드뿐 아니라 자기 자신의 정보도 함께 반영하기 위해 self-loop를 추가함
A_hat = A + I

# degree 계산
# 각 노드가 몇 개의 연결을 갖는지 계산
degree = A_hat.sum(dim=1)

#  $D^{-1/2}$ 
D_inv_sqrt = torch.diag(torch.pow(degree, -0.5))

# 정규화된 인접행렬
# GCN 기본 정규화:  $D^{-1/2} A_{\text{hat}} D^{-1/2}$ -연결 수가 많은 노드의 영향이 과도하게 커지는 것을 막기 위한 과정
A_norm = D_inv_sqrt @ A_hat @ D_inv_sqrt
```

☑ 그래프 합성곱 네트워크(Graph Convolution Network) 실습 : 모델 준비

```
class GCNLayer(nn.Module):
    def __init__(self, in_features, out_features):
        super(GCNLayer, self).__init__()

        # 노드 feature를 변환하는 학습 가능한 선형층
        self.linear = nn.Linear(in_features, out_features)

    def forward(self, X, A_norm):
        # 1. 인접행렬을 이용해 이웃 노드 정보 집계
        # shape: [num_nodes, in_features]
        X = torch.matmul(A_norm, X)

        # 2. 선형 변환
        # shape: [num_nodes, out_features]
        X = self.linear(X)

        return X
```

```
class SimpleGraphClassifier(nn.Module):
    def __init__(self, node_in_dim, hidden_dim, graph_feature_dim, num_classes):
        super(SimpleGraphClassifier, self).__init__()

        # 그래프 합성곱층
        self.gcn1 = GCNLayer(node_in_dim, hidden_dim)
        self.gcn2 = GCNLayer(hidden_dim, hidden_dim)

        # Readout 이후 그래프 특성과 결합한 뒤 분류
        self.fc = nn.Linear(hidden_dim + graph_feature_dim, num_classes)

    def forward(self, X, A_norm, graph_feature):
        # 첫 번째 GCN layer
        h = self.gcn1(X, A_norm)
        h = F.relu(h)

        # 두 번째 GCN layer
        h = self.gcn2(h, A_norm)
        h = F.relu(h)

        # Readout
        # 노드별 표현 h를 그래프 하나의 표현으로 요약
        # 여기서는 mean pooling 사용
        graph_embedding = h.mean(dim=0)

        # 그래프 특성과 GCN으로 얻은 그래프 임베딩을 결합
        combined = torch.cat([graph_embedding, graph_feature], dim=0)

        # 완전연결층을 통해 최종 분류
        out = self.fc(combined)

        # CrossEntropyLoss에 넣기 위해 batch 차원 추가
        out = out.unsqueeze(0)

        return out
```

☑ 그래프 합성곱 네트워크(Graph Convolution Network) 실습 : 모델 준비

```
model = SimpleGraphClassifier(
    node_in_dim=3,
    hidden_dim=8,
    graph_feature_dim=2,
    num_classes=2
)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

```
loss_history = []
acc_history = []
```

```
for epoch in range(200):
    model.train()
```

```
    # forward
    output = model(X, A_norm, graph_feature)
```

```
    # loss 계산
    loss = criterion(output, label)
```

```
    # gradient 초기화
    optimizer.zero_grad()
```

```
    # backward
    loss.backward()
```

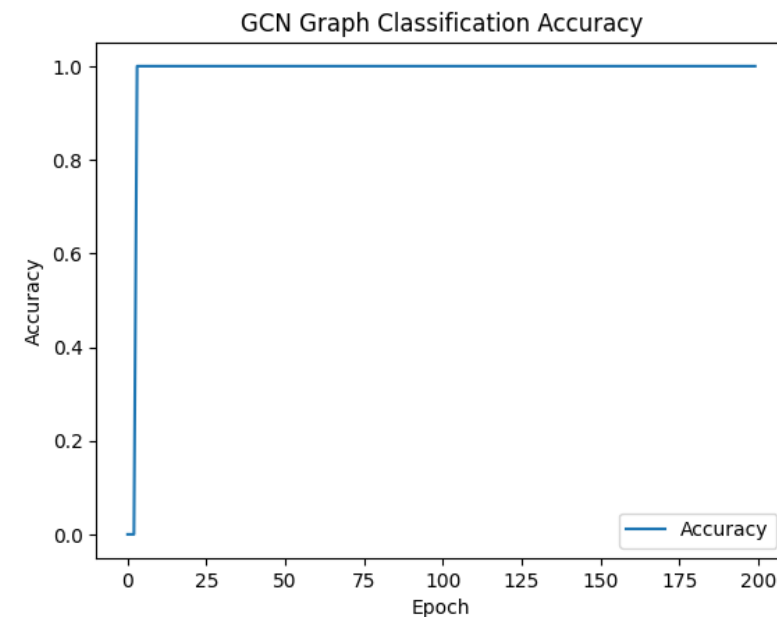
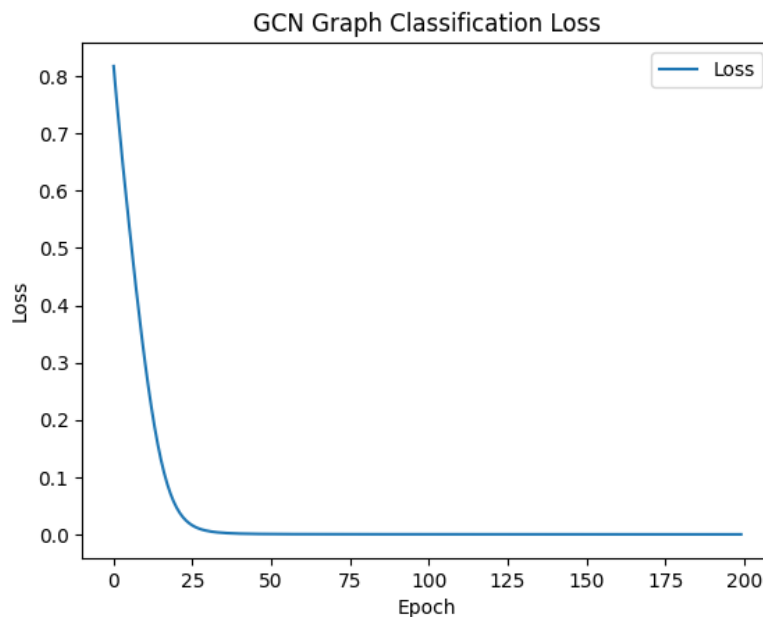
```
    # parameter update
    optimizer.step()
```

```
    # prediction
    _, pred = torch.max(output, dim=1)
    acc = (pred == label).float().mean()
```

```
    loss_history.append(loss.item())
    acc_history.append(acc.item())
```

```
    if epoch % 20 == 0:
        print(f"Epoch {epoch:03d} | Loss: {loss.item():.4f} | Acc: {acc.item():.4f}")
```

```
Epoch 000 | Loss: 0.8177 | Acc: 0.0000
Epoch 020 | Loss: 0.0465 | Acc: 1.0000
Epoch 040 | Loss: 0.0015 | Acc: 1.0000
Epoch 060 | Loss: 0.0005 | Acc: 1.0000
Epoch 080 | Loss: 0.0003 | Acc: 1.0000
Epoch 100 | Loss: 0.0003 | Acc: 1.0000
Epoch 120 | Loss: 0.0002 | Acc: 1.0000
Epoch 140 | Loss: 0.0002 | Acc: 1.0000
Epoch 160 | Loss: 0.0002 | Acc: 1.0000
Epoch 180 | Loss: 0.0001 | Acc: 1.0000
```



감사합니다